



SRE

SOFTWARE REQUIREMENT ENGINEERING

COMPREHENSIVE STUDY NOTES

Prepared & Curated By

Shaikh Mahad

[linkedin.com/in/codewithmahad](https://www.linkedin.com/in/codewithmahad)

github.com/codewithmahad

Course Information & Outline

Course Code:	SE-453
Course Title:	Software Requirement Engineering
Instructor:	Sir Hussain (Assistant Professor)
Credit Hours:	3+0 (Theory: 03 Lab: 00)

Topics Covered in this Document:

1. Introduction to Requirements Engineering
2. Software Requirements, classification of requirements
3. Requirements process, Levels/layers of requirements
4. Requirement characteristics, Analyzing quality requirements
5. Software requirements in the context of systems engineering
6. Requirement evolution, requirement traceability, requirement prioritization
7. Trade-off analysis
8. Risk analysis and impact analysis
9. Requirement management
10. Interaction between requirement and architecture
11. Requirement elicitation
12. Elicitation sources and techniques
13. Requirement specification and documentation
14. Specification sources and techniques
15. Requirements validation and techniques
16. Management of Requirements
17. Introduction to Management
18. Requirements Management Problems
19. Managing Requirements in an Acquisition Organization
20. Supplier Organizations
21. Product Organizations
22. Requirements engineering for agile methods

Table of Contents

Course Information & Outline	1
1 Introduction to Requirement Engineering	5
1.1 Introduction	5
1.2 What are Software Requirements?	5
1.3 Importance of Requirement Engineering	5
1.4 Stakeholders in Requirements Engineering	5
1.5 Types of Software Requirements	6
1.5.1 Functional Requirements	6
1.5.2 Non-Functional (Quality) Requirements	6
1.5.3 Domain Requirements	6
1.6 User Requirements vs. System Requirements	7
1.7 Case Study & Process Flow	7
2 Requirements Process and Analysis	9
2.1 Requirements Engineering Process	9
2.2 Major Activities in the Requirements Process	9
2.2.1 Requirements Elicitation	9
2.2.2 Requirements Analysis	9
2.2.3 Requirements Specification	10
2.2.4 Requirements Validation	10
2.3 Characteristics of Good Requirements	10
2.4 Analyzing Quality Requirements	11
3 Software Requirements in Systems Engineering Context	12
3.1 The Software Context	12
3.2 Relationship Between System and Software Requirements	12
3.3 Requirement Evolution	13
3.4 Types of Changing Requirements	13
3.5 Requirements Traceability	14
3.6 Requirements Prioritization	14
3.7 Importance of Requirement Management	14
4 Requirement Prioritization and Trade-off Analysis	15
4.1 Importance of Requirement Prioritization	15
4.2 Factors Affecting Requirement Priority	15
4.3 Techniques for Requirement Prioritization	15
4.4 Trade-off Analysis	16
4.5 Trade-off Analysis Process	16
5 Risk Analysis and Impact Analysis	17
5.1 Risk Analysis	17
5.2 Types of Software Risks	17
5.3 Risk Identification & Assessment	17
5.4 Risk Mitigation	18
5.5 Impact Analysis	18

6	Requirements and Architecture	19
6.1	Introduction	19
6.2	What is Software Architecture?	19
6.3	Interaction Between Requirements and Architecture	19
6.4	Importance of Aligning Requirements & Architecture	19
6.5	Requirements Elicitation (Revisited)	19
6.6	Stakeholders in Elicitation	20
6.7	Elicitation Process	20
6.8	Challenges in Requirements Elicitation	20
6.9	Summary of Week 6	20
7	Requirement Sources and Specification	21
7.1	Sources of Requirements	21
7.2	Requirements Elicitation Techniques	21
7.2.1	Interviews	21
7.2.2	Questionnaires	21
7.2.3	Observation	21
7.2.4	Workshops	21
7.2.5	Prototyping	22
7.3	Requirement Specification	22
7.4	Documentation of Requirements	22
8	Specification Techniques and Validation	23
8.1	Specification Techniques	23
8.2	Requirements Validation	23
8.3	Requirements Validation Techniques	23
9	Management of Requirements	25
9.1	Requirements Management	25
9.2	Activities in Requirements Management	25
9.3	Change Management Process	25
9.4	Requirement Traceability	25
9.5	Introduction to Management in SRE	25
10	Management Problems & Acquisition Organizations	27
10.1	Requirements Management Problems	27
10.2	Acquisition Organizations	27
10.3	Challenges in Acquisition Organizations	27
10.4	Managing Requirements in Acquisition Organizations	27
11	Supplier and Product Organizations	28
11.1	Introduction	28
11.2	Supplier Organizations	28
11.3	Product Organizations	28
12	Requirements Engineering for Agile Methods	29
12.1	Introduction	29
12.2	Agile Requirements	29
12.3	Agile Requirements Process	29
12.4	Advantages of Agile Requirements	29

12.5 Challenges of Agile Requirements	29
---	----

1 Introduction to Requirement Engineering

1.1 Introduction

Software development begins by understanding what the system should do and what the users expect from it. This fundamental understanding is captured through **requirements**.

Definition: Requirements Engineering

Requirements Engineering is the process of discovering, analyzing, documenting, and managing the needs and expectations of stakeholders for a software system.

It plays a critical role in the success of software projects because incorrect or incomplete requirements can lead to system failure. The main goal of requirements engineering is to ensure that the system being developed satisfies the actual needs of its users and stakeholders.

1.2 What are Software Requirements?

Software requirements describe the services that a system should provide and the constraints under which the system must operate. These requirements reflect the needs of the customers and the purpose of the system.

In simple terms:

- Requirements explain **what** a system should do and **how** it should behave.
- They act as a crucial bridge between the customer and the development team.

1.3 Importance of Requirement Engineering

Requirements engineering is essential for several critical reasons:

- It helps developers understand what the customer actually wants.
- It reduces misunderstandings between developers and clients.
- It ensures that the final software system satisfies user needs.
- It helps avoid costly mistakes during the later stages of development.
- It provides a clear foundation for system design and development.

If requirements are not properly defined, the software system may ultimately fail to meet user expectations.

1.4 Stakeholders in Requirements Engineering

Stakeholders are individuals or groups who have an interest in the system or are directly/indirectly affected by it. Each stakeholder may have different expectations from the system, so it is vital to gather requirements from all relevant parties.

Examples of stakeholders include:

- End Users
- Customers or Clients
- System Developers
- Project Managers
- System Testers
- Maintenance Engineers

1.5 Types of Software Requirements

Software requirements are generally classified into three main categories: Functional, Non-Functional, and Domain requirements.

1.5.1 Functional Requirements

These describe the services or functions that the system must perform. They specify what the system should do in response to specific inputs or situations. They focus entirely on the *behavior* of the system.

- The system shall allow users to create accounts.
- The system shall allow users to search for products.
- The system shall generate daily reports.

1.5.2 Non-Functional (Quality) Requirements

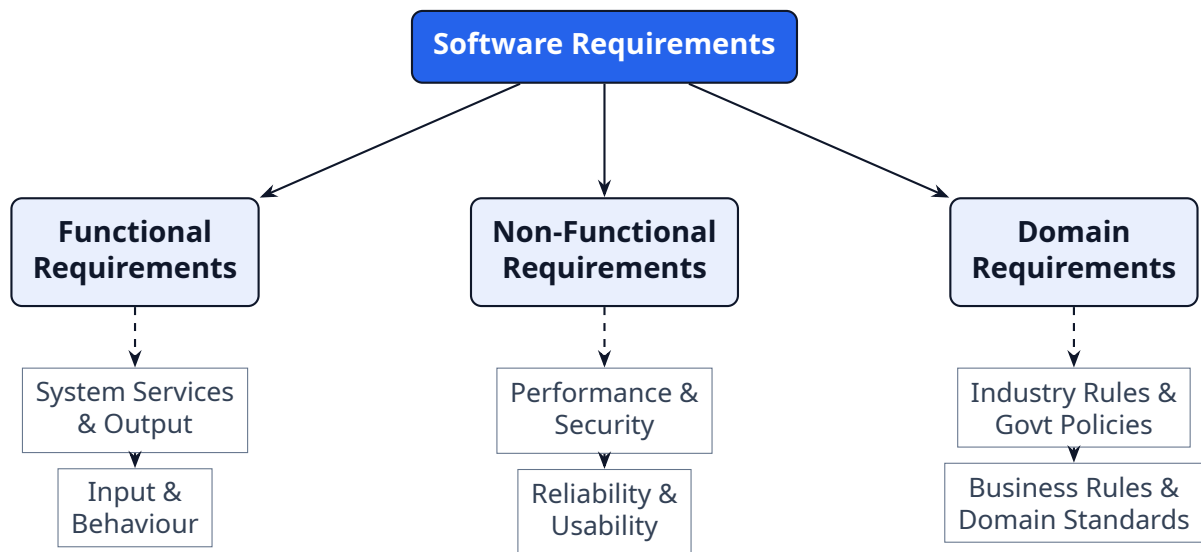
These describe the constraints and quality attributes of the system. They define *how well* the system should perform rather than what the system should do. Non-functional requirements often apply to the entire system rather than individual features.

- The system should respond within 2 seconds.
- The system should be secure and protect user data.
- The system should be available 24/7.

1.5.3 Domain Requirements

These requirements come from the application domain of the system, not directly from the users. They depend on the specific field where the system will be used and may introduce new functional requirements or place constraints on existing ones.

- Medical systems must follow strict healthcare regulations.
- Banking systems must adhere to financial security rules.



1.6 User Requirements vs. System Requirements

Requirements can also be categorized based on their level of detail and their intended audience.

- **User Requirements:** Describe the system services in a simple, understandable form. They are usually written in natural language and simple diagrams so that non-technical stakeholders (managers, clients) can understand them.
Example: The system should allow users to upload photos.
- **System Requirements:** Provide a highly detailed description of system functions and constraints. They are mainly used by developers and engineers.
Example: The system shall allow users to upload images in JPG or PNG format with a maximum size of 5 MB.

1.7 Case Study & Process Flow

Example Case Study: Online Shopping System

Functional Requirements:

- Users can register and login.
- Users can search for products.
- Users can place orders.

Non-Functional Requirements:

- The system should load pages within 3 seconds.
- The system must protect customer payment data.
- The system should support thousands of users simultaneously.



2 Requirements Process and Analysis

2.1 Requirements Engineering Process

The requirements engineering process is a structured set of activities used to discover, analyze, document, and validate the requirements of a software system. This process helps developers understand what the system should do and ensures that the final product satisfies the needs of stakeholders.

These activities are **not always performed in a strict sequence**. In practice, they are iterative and interleaved, meaning that developers may move back and forth between activities as their understanding of the system improves. The final output of this process is a document called the **Software Requirement Specification (SRS)**, which describes the system requirements in detail.

2.2 Major Activities in the Requirements Process

2.2.1 Requirements Elicitation

It is the process of gathering information from stakeholders about what they want to build. During this stage, software engineers interact with users, managers, and other stakeholders to understand:

- The services the system should provide.
- The expected performance of the system.
- Constraints such as hardware, budget, or regulations.

Note: Eliciting requirements is often difficult because stakeholders may not clearly know what they want or may express their needs in vague terms.

2.2.2 Requirements Analysis

This involves examining and organizing the collected requirements to understand them clearly. During analysis, engineers:

- Identify conflicts between requirements.
- Organize requirements into logical groups.
- Prioritize requirements based on importance.
- Resolve inconsistencies between stakeholders.

This stage ensures that the requirements are clear, complete, and understandable before official documentation begins.

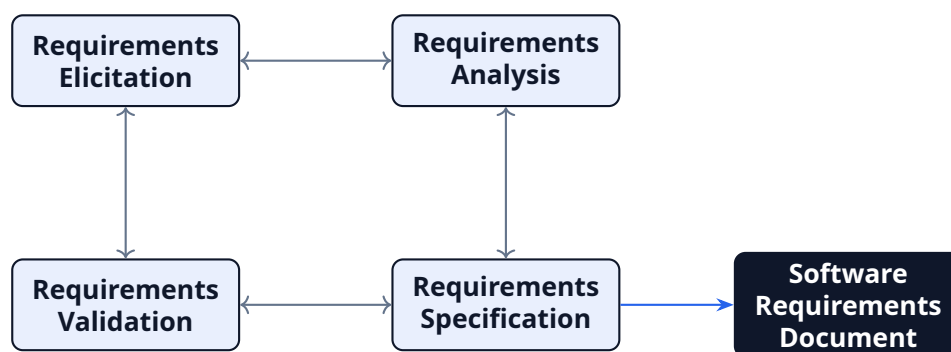
2.2.3 Requirements Specification

It is the process of documenting the requirements in a clear and structured way. The result of this stage is the SRS document, which acts as a formal agreement between the client and the development team. The SRS includes:

- Functional and Non-functional requirements.
- System constraints, diagrams, and models.

2.2.4 Requirements Validation

It ensures that the documented requirements correctly represent the needs of stakeholders. Engineers check the requirements for: **Correctness, Completeness, Consistency, and Feasibility**. Validation is crucial because errors in requirements can lead to exceptionally expensive mistakes during development.



2.3 Characteristics of Good Requirements

For requirements to be effective and useful, they must possess certain vital characteristics:

1. **Correct:** Accurately describes the functionality stakeholders need.
2. **Complete:** Includes all necessary system services and constraints. Incomplete requirements cause system failures.
3. **Consistent:** Requirements should not conflict with each other (e.g., demanding lightning-fast response while simultaneously requiring heavy, slow processing).
4. **Verifiable:** Must be measurable or testable so developers can verify correct implementation.
5. **Feasible:** Must be realistic and achievable within available time, budget, and technology constraints.
6. **Traceable:** Should be traceable to its source so developers know *why* it exists and how it relates to others. Crucial for impact analysis.

2.4 Analyzing Quality Requirements

Quality requirements (Non-Functional Requirements) define the quality attributes of a system. They apply to the entire system rather than individual features and determine whether the system is ultimately practical and usable.

Examples of Core Quality Requirements

- **Performance:** The system should process at least 100 transactions per second.
- **Reliability:** The system should maintain 99.9% availability.
- **Usability:** Users should be able to learn basic operations within 30 minutes of training.
- **Security:** Only authorized users should be able to access confidential data.
- **Maintainability:** The system should be easy to update without affecting other components.

Importance: If performance requirements aren't satisfied, the system becomes too slow. If security is ignored, sensitive information is exposed. In many cases, failing to meet non-functional requirements renders the entire system effectively useless.

3 Software Requirements in Systems Engineering Context

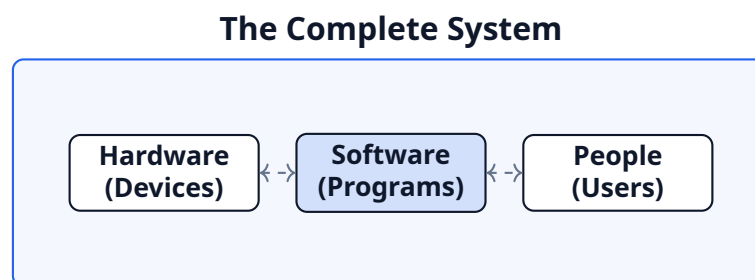
3.1 The Software Context

In large projects, software is rarely developed in isolation. It is usually part of a much larger ecosystem that includes hardware, software, people, procedures, and data.

Systems Engineering is the discipline that focuses on designing and managing these complex systems throughout their lifecycle. In this context, software requirements must be considered together with other system components to ensure the entire system functions effectively. A software system may interact with:

- Hardware devices
- Databases
- Other software systems
- Communication networks
- Human users

Therefore, requirements engineering must heavily consider these interactions when defining system behavior.



3.2 Relationship Between System and Software Requirements

System requirements describe the overall behavior of the complete system, while software requirements describe only the specific behavior of the software component residing *within* that system. Thus, software requirements are a strict **subset** of system requirements.

Example: ATM Machine Architecture

System Requirements (The Whole):

- ATM Machine Hardware (Screen, Keypad)
- Network Connection
- Cash Dispenser Device
- Banking Software

Software Requirements (The Subset):

- Authentication of users via PIN.
- Transaction processing and account balance checking.
- The logical money transfer function.

3.3 Requirement Evolution

Requirements do **not** remain fixed throughout the life of a system. They constantly change and evolve over time because organizations grow, technology improves, and users discover new needs.

A system's environment is always changing due to:

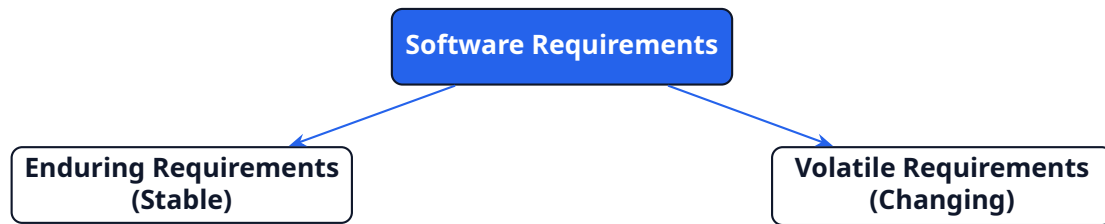
- New business policies.
- Improvements in hardware or technology.
- New laws, compliance, or government regulations.
- Feedback from users after system deployment.
- Integration with new external systems.

Because of these changes, requirements evolve both *during* system development and *after* the system has launched. As developers gain more knowledge about the system and user needs, the requirements are continually refined and updated.

3.4 Types of Changing Requirements

Requirements can be categorized based on how frequently they change:

- **Enduring Requirements (Stable):** These are stable requirements that rarely change. They are usually related to the core activities of an organization.
Example: Banks must record financial transactions; hospitals must store patient records.
- **Volatile Requirements (Changing):** These are more likely to change frequently. They are usually related to user interface design, reporting formats, or business rules as organizations update their processes.



3.5 Requirements Traceability

Requirements traceability is the ability to track the relationships between requirements and other system elements. It allows engineers to understand:

- Where a requirement originated.
- How it relates to other requirements.
- How it affects system design and implementation.

Traceability is extremely important when requirements change because developers must understand how the change will affect other parts of the system.

Example: If the requirement "User must login before accessing the system" changes, it may affect the authentication module, database user records, and security policies.



3.6 Requirements Prioritization

In most software projects, it is impossible to implement all requirements at the same time because of limited time, budget, and resources. Therefore, requirements must be prioritized so that the most important features are developed first.

Common Priority Levels:

- **High Priority:** Essential features that must be implemented for the system to function (e.g., user authentication in a banking system).
- **Medium Priority:** Important features but not critical for the first release (e.g., advanced search filters).
- **Low Priority:** Features that are desirable but can be implemented later (e.g., custom UI themes).

3.7 Importance of Requirement Management

Since requirements change frequently, organizations must manage these changes effectively. Requirements management involves identifying requirements, tracking changes, and controlling updates. A formal change management process ensures that modifications are analyzed carefully before implementation.

4 Requirement Prioritization and Trade-off Analysis

4.1 Importance of Requirement Prioritization

Requirement prioritization is important for several reasons:

- It helps teams deliver critical features first.
- It ensures efficient use of development resources.
- It supports better planning and scheduling.
- It reduces project risks.

Without prioritization, development teams may waste time implementing less important features while critical features remain unfinished.

4.2 Factors Affecting Requirement Priority

Several factors influence how requirements are prioritized:

- **Business Value:** Requirements that provide greater business benefits should be implemented earlier.
- **Implementation Cost:** Engineers must evaluate whether the benefits of a feature justify its development effort and cost.
- **Risk Level:** High-risk requirements should often be addressed early so potential problems can be solved.
- **Stakeholder Importance:** Different stakeholders have different priorities, and their influence affects the final prioritization.

4.3 Techniques for Requirement Prioritization

1. The MoSCoW Method

A common technique that divides requirements into four categories:

- **Must Have (Critical)**
- **Should Have (Important)**
- **Could Have (Optional/Desirable)**
- **Won't Have (Future Release)**

- **2. Numerical Ranking:** Each requirement is assigned a numerical value (e.g., 1 to 10) based on its importance. Higher numbers indicate higher priority.
- **3. Stakeholder Voting:** Stakeholders vote on the requirements they consider most important. Requirements receiving the highest votes are given priority.

4.4 Trade-off Analysis

Trade-off analysis is the process of evaluating conflicting requirements or design choices in order to determine the best solution. In software systems, improving one quality attribute may negatively affect another.

- Increasing security may reduce system usability.
- Improving performance may increase development cost.
- Adding new features may increase system complexity.

Example: Mobile Banking Application

Option A: High security with multi-factor authentication. (Better security, but reduces user convenience).

Option B: Simple login process with faster access. (Improves usability, but reduces security strength).

Engineers must analyze these trade-offs to determine which option best satisfies the system's overarching goals.

4.5 Trade-off Analysis Process

The analysis usually involves the following structured steps:

1. Identify conflicting requirements.
2. Evaluate the impact of each option.
3. Analyze benefits and drawbacks (Advantages & Disadvantages).
4. Select the most balanced solution.



5 Risk Analysis and Impact Analysis

5.1 Risk Analysis

Risk analysis is the process of identifying potential problems that could negatively affect the success of a software project. In software engineering, risks may arise due to:

- Unclear requirements
- Changing requirements
- Technical complexity
- Limited resources
- Unrealistic deadlines

Risk analysis helps project teams anticipate problems early and prepare solutions in advance.

5.2 Types of Software Risks

Risks in software projects can generally be classified into three main categories:

- **Project Risks:** Affect the project schedule, budget, or resources. (e.g., lack of skilled developers, delays in schedule, insufficient funding).
- **Technical Risks:** Arise due to technical challenges. (e.g., use of unfamiliar technology, complex architecture, integration difficulties).
- **Business Risks:** Affect the organization's ability to benefit from the system. (e.g., system not meeting market needs, loss of customer trust, project cancellation).



5.3 Risk Identification & Assessment

The first step is identifying potential risks using techniques like brainstorming sessions, expert consultation, and reviewing past project experiences. Once identified, engineers evaluate:

1. The **probability** that the risk will occur.
2. The potential **impact** if the risk occurs.

Risks with high probability and high impact require immediate attention.

5.4 Risk Mitigation

Risk mitigation refers to strategies used to reduce or eliminate risks. Examples include:

- Using proven technologies instead of experimental ones.
- Conducting regular project reviews.
- Improving communication among stakeholders.
- Creating backup plans for critical tasks.

5.5 Impact Analysis

Impact analysis is the process of analyzing the consequences of a requirement change. When a requirement changes, it may affect several components such as the system design, database structure, program code, and testing procedures.

Importance of Impact Analysis:

- Prevents unexpected system failures.
- Helps estimate development cost and time.
- Ensures that system changes are properly managed and reduces risks.

6 Requirements and Architecture

6.1 Introduction

In software development, requirements and system architecture are closely related. **Requirements** describe *what* the system should do, while **architecture** describes *how* the system will be structured to fulfill those requirements. A well-designed architecture ensures that the system satisfies both functional and non-functional requirements.

6.2 What is Software Architecture?

Software architecture refers to the overall structure of a software system. It acts as a blueprint and defines:

- Major system components.
- Relationships between components.
- Communication mechanisms.
- Overall system organization.

6.3 Interaction Between Requirements and Architecture

Requirements influence the design of the architecture, while architecture may also influence how requirements are implemented.

- **Performance requirements** affect system processing design.
- **Security requirements** influence authentication mechanisms.
- **Scalability requirements** affect deployment architecture.

Because of this tight coupling, requirements and architecture must be designed together.

6.4 Importance of Aligning Requirements & Architecture

Aligning them ensures the system design satisfies user needs, improves performance and reliability, reduces errors, and simplifies future maintenance. If the architecture does not support the requirements, the system will ultimately fail to meet stakeholder expectations.

6.5 Requirements Elicitation (Revisited)

Requirements elicitation is the process of collecting system requirements from stakeholders and other sources. It is one of the most critical stages because incorrect or incomplete requirements directly lead to system failure. The main goal is to fully understand the needs, expectations, and constraints of all stakeholders before architectural design solidifies.

6.6 Stakeholders in Elicitation

Stakeholders are individuals or groups who are directly or indirectly affected by the system. Each stakeholder may have different expectations from the system. Examples include:

- Customers
- Project Managers
- System Users
- Business Managers
- Developers
- System Administrators

6.7 Elicitation Process

The requirement elicitation process generally includes the following sequential steps:

1. **Identify** stakeholders.
2. **Gather** information about system needs.
3. **Analyze** the collected requirements.
4. **Resolve** conflicts between requirements.
5. **Document** the requirements.

6.8 Challenges in Requirements Elicitation

Requirements elicitation may face several significant challenges:

- **Communication Problems:** Stakeholders may not clearly explain their needs.
- **Conflicting Requirements:** Different stakeholders may have different expectations.
- **Changing Requirements:** Stakeholders' needs may change during development.
- **Incomplete Knowledge:** Sometimes users do not fully understand their own requirements.

6.9 Summary of Week 6

Requirements and architecture are strongly interconnected. Requirements define *what* the system must achieve, while architecture provides the *structural design* to implement those requirements. Requirements elicitation is the process of gathering system requirements from stakeholders and other sources, playing a critical role in ensuring the success of a software project.

7 Requirement Sources and Specification

7.1 Sources of Requirements

Requirements may originate from several sources within an organization. Common sources include:

- **System Users:** Users interact directly with the system and provide valuable information about required features.
- **Customers:** Customers specify overarching business goals and expectations.
- **Business Processes:** Existing business processes often dictate mandatory system requirements.
- **Existing Systems:** Older, legacy systems may provide insight into required functionality and constraints.

7.2 Requirements Elicitation Techniques

Software engineers use various techniques to gather requirements from stakeholders depending on the system's complexity and stakeholder availability.

7.2.1 Interviews

Interviews involve direct communication between developers and stakeholders. They can be structured, semi-structured, or unstructured.

Advantages: Provides detailed information and allows for direct clarification of requirements.

7.2.2 Questionnaires

Questionnaires are useful when information must be collected from a large number of stakeholders.

Advantages: Quick data collection and easy to distribute globally.

7.2.3 Observation

Observation involves watching users perform their tasks in real environments.

Advantages: Identifies hidden requirements and heavily improves understanding of actual user behavior.

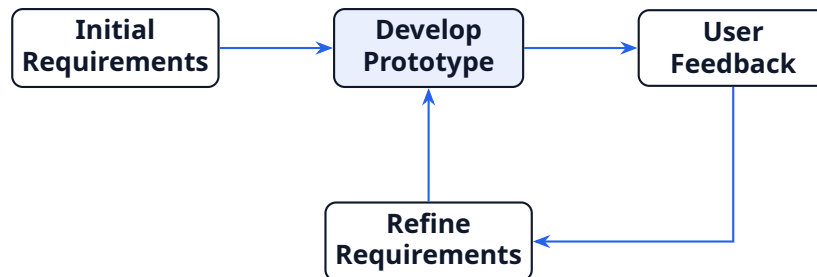
7.2.4 Workshops

Workshops involve group discussions between stakeholders and developers.

Benefits: Encourages collaboration and rapidly resolves requirement conflicts.

7.2.5 Prototyping

Prototyping involves building a preliminary model of the system. Stakeholders can interact with the prototype and provide targeted feedback.



7.3 Requirement Specification

Requirement specification is the process of documenting system requirements clearly and formally. The document that contains all system requirements is called the **Software Requirements Specification (SRS)**. The SRS document serves as the ultimate reference for developers, testers, and project managers.

Purpose of SRS:

- Provides a detailed system description.
- Ensures a clear understanding of system functionality.
- Acts as a formal agreement between stakeholders & developers.
- Provides strict guidance for system design & development.

Characteristics of a Good SRS

A high-quality SRS document should be:

- | | |
|--------------|---------------|
| • Correct | • Modifiable |
| • Consistent | • Traceable |
| • Complete | • Unambiguous |
| • Verifiable | |

7.4 Documentation of Requirements

Requirements documentation ensures that all stakeholders understand the system clearly. It usually includes: a system overview, functional requirements, non-functional requirements, constraints, and system models/diagrams.

8 Specification Techniques and Validation

(Note: Week 8 generally covers Course Review / Midterms)

8.1 Specification Techniques

Requirement specification techniques are used to represent requirements clearly and systematically so that all stakeholders can understand them. Different techniques are used depending on system complexity.

- **Natural Language Specification:** Requirements are written in plain English.
Example: "The system shall allow users to log in using username and password."
Pros: Easy to understand, no technical knowledge required. **Cons:** Prone to ambiguity, difficult to verify mathematically.
- **Structured Specification:** Requirements are written in a structured format using templates (e.g., Requirement ID, Description, Inputs, Outputs, Constraints). This reduces ambiguity and improves clarity.
- **Graphical Models:** Visual representations help visualize system behavior. Examples include Data Flow Diagrams (DFD), Use Case Diagrams, and UML Diagrams.
- **Mathematical Specification:** Uses mathematical expressions to define system behavior.
Pros: Precise and unambiguous. **Cons:** Very difficult for non-technical stakeholders to understand.

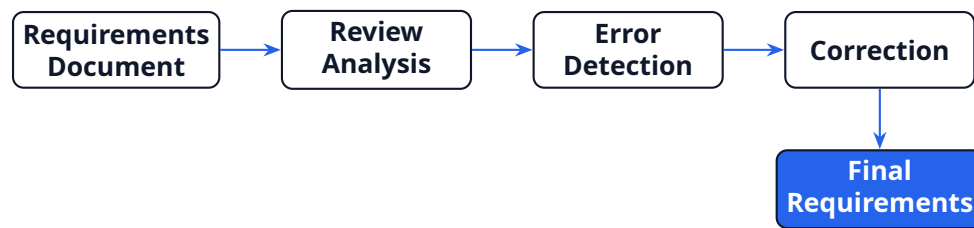
8.2 Requirements Validation

Requirements validation is the process of ensuring that requirements are correct, complete, and meet user needs. It answers the most important software engineering question: **"Are we building the right system?"**

Importance of Validation: Errors are much cheaper to fix early in the lifecycle. Thorough validation prevents system failure, ensures customer satisfaction, and improves overall software quality.

8.3 Requirements Validation Techniques

1. **Requirements Reviews:** Stakeholders and developers systematically read and analyze the requirements document to identify errors, missing requirements, or inconsistencies.
2. **Prototyping:** An early model of the system is developed and shown to users. User feedback directly helps refine the requirements.
3. **Test Case Generation:** Test cases are created based on requirements to check if they are actually testable. If a requirement cannot be tested, it is likely unclear.
4. **Automated Validation Tools:** Software tools are used to check consistency, detect missing requirements, and validate structural integrity.



9 Management of Requirements

9.1 Requirements Management

Requirements management is the process of handling changes in requirements throughout the software lifecycle. Requirements may change due to new user needs, business environment changes, or technical constraints. Managing these changes properly is absolutely essential for project success.

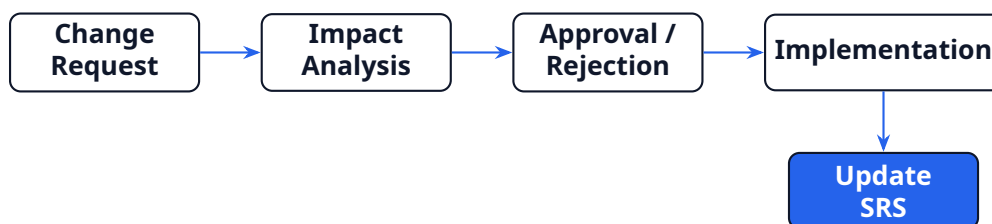
9.2 Activities in Requirements Management

Key activities include:

- Change Management
- Requirement Tracking
- Version Control
- Impact Analysis

9.3 Change Management Process

When a requirement changes, it must go through a proper, formal process to ensure system stability.



9.4 Requirement Traceability

Traceability is the ability to track requirements throughout the development process. It heavily assists developers by answering:

- Where a requirement is used.
- How it is implemented in the codebase.
- What happens to the system if it changes.

9.5 Introduction to Management in SRE

Management in software engineering refers to planning, organizing, and controlling software development activities. It is a broad discipline that encompasses:

- Project Planning
- Resource Management

- Risk Management
- Quality Control

Effective requirements management handles changing requirements gracefully, improves system quality, reduces project risks, and ensures the project stays on track.

10 Management Problems & Acquisition Organizations

10.1 Requirements Management Problems

Managing requirements is frequently challenging due to several common problems:

- **Changing Requirements:** Requirements may frequently change, causing instability.
- **Conflicting Requirements:** Different stakeholders may hold different expectations.
- **Lack of Communication:** Poor communication inevitably leads to misunderstandings.
- **Incomplete Requirements:** Some requirements may simply be missing or extremely unclear.

Managing These Problems: Solutions include proper documentation, regular stakeholder communication, use of specialized management tools, and strict change control processes.

10.2 Acquisition Organizations

An acquisition organization is one that purchases or outsources software development instead of developing the software internally. They hire external vendors to build their systems.

10.3 Challenges in Acquisition Organizations

- **Communication Gap:** The client and the external supplier may deeply misunderstand requirements.
- **Contract Issues:** Requirements must be clearly defined in legally binding contracts.
- **Requirement Changes:** Changes are extremely difficult (and expensive) to execute after a contract agreement is signed.
- **Dependency:** The client relies heavily on an external supplier.

10.4 Managing Requirements in Acquisition Organizations

To mitigate these risks, organizations must enforce a **Clear Contract Specification** where all requirements are well-defined. They must maintain **Regular Communication** (frequent meetings with the supplier), enforce a **Change Control System** (formal approval required for changes), and heavily **Monitor Progress** to track development.



11 Supplier and Product Organizations

11.1 Introduction

In software engineering, organizations develop systems in different business models. The two major types are **Supplier Organizations** and **Product Organizations**. Understanding the difference is important because it dictates how requirements are gathered, managed, and changed over time.

11.2 Supplier Organizations

Supplier organizations develop software for external clients. They work strictly based on contracts and specific customer requirements.

- Work for multiple distinct clients.
- Follow strict, legally binding contracts.
- Heavily focused on delivery deadlines.

Challenges: Unclear client requirements, communication issues, and handling changing requirements mid-contract.

11.3 Product Organizations

Product organizations develop software products for the open market instead of specific clients (e.g., mobile apps, SaaS platforms, commercial software).

- Focus on broad market needs.
- Continuous product improvement based on analytics.
- Long-term development lifecycles.

Comparison: Supplier vs. Product Organization

Supplier Organization

- Change is difficult
- Client-based
- Contract-driven
- Fixed requirements
- Short-term projects

Product Organization

- Change is flexible
- Market-based
- Product vision-driven
- Evolving requirements
- Long-term projects / dev

12 Requirements Engineering for Agile Methods

12.1 Introduction

Agile methods focus heavily on flexibility and continuous change in software development. Unlike traditional waterfall methods, requirements are not entirely fixed at the beginning of the project.

12.2 Agile Requirements

In agile development, requirements are written as **User Stories**, they continually evolve over time, and continuous feedback is extracted directly from the users.

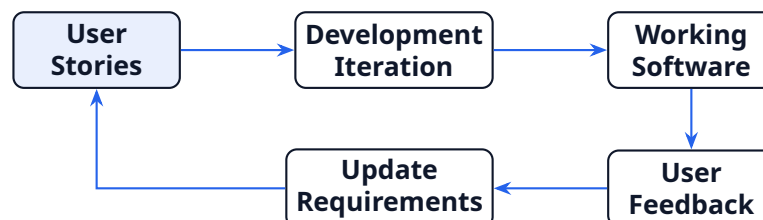
Example: Agile User Story Format

"As a [user type], I want to [action/goal] so that [benefit/value]."

Actual Example: As a user, I want to log in so that I can securely access my personal account.

Characteristics of Agile Requirements: Simple and short, highly flexible, and intensely focused on actual user needs rather than system technicalities.

12.3 Agile Requirements Process



12.4 Advantages of Agile Requirements

- Handles changing requirements easily and gracefully.
- Drastically improves customer satisfaction due to inclusion in the process.
- Allows for faster delivery of high-value features.
- Reduces overall project risk.

12.5 Challenges of Agile Requirements

Despite the benefits, Agile poses challenges such as a lack of detailed system documentation, difficulty in managing incredibly large or safety-critical systems, and the necessity for extremely strong, continuous communication among the team.

A Note from a Fellow Student

Assalam-o-Alaikum!

I am **Shaikh Mahad**, a student of UBIT (2025 Batch). I originally compiled these notes during my Third semester while studying Software Requirement Engineering (SE-453).

When I was taking this course, I struggled to find well-structured learning materials. Rather than keeping these notes to myself, I compiled them into this document to leave behind a valuable, free resource for all future juniors. This PDF is designed to help you genuinely understand the core concepts and master the topics with ease. I hope it saves you time and makes your academic journey much smoother.

Get All My Notes & Resources

Instead of messaging me individually, you can find all my PDFs, study materials, and code resources systematically organized on my GitHub. If you find these materials helpful, please **Star the repository** and **Follow me on GitHub** to stay updated whenever I drop new content!

[Access & Star the GitHub Repository Here](#)

Join The UBIT Hub Community

Let's grow together! The UBIT Hub is a dedicated WhatsApp community built specifically for our students to share high-quality notes and help everyone succeed.

[Click Here to Join The UBIT Hub on WhatsApp](#)

Let's Connect:

LinkedIn: [linkedin.com/in/codewithmahad](https://www.linkedin.com/in/codewithmahad)

GitHub: github.com/codewithmahad